Software Engineering Lab
(25CS60TP1379-1)

M.C.A. (AI & ML) Semester – III



Session - Winter 2026-27

Ramdeobaba University, Nagpur
Department of Computer Science and Applications

Prepared By

Sejal Kashyap
Vinay Chalsani
Tejaswini Kadu
Vidya Gupta

Under the Guidance of

Professor Sudhanshu Tripathi

1.Software Requirements Specification

CareBridge: A Smart Need-to-Impact Platform for NGOs

1.1. Project title and introduction

CareBridge – A Smart Need-to-Impact Platform for NGOs is a responsive web application designed to connect NGOs, donors, supporters, volunteers, CSR partners, and beneficiaries through a transparent lifecycle. The system focuses on specific community needs rather than only campaign-level fundraising. Its intended lifecycle is to identify need, prioritize, match, support, track, verify, and measure impact.

The supplied application is a TypeScript/React frontend using TanStack Start/Router and Vite, with demo records in src/lib/mock-data.ts. The SRS describes intended capability; the archive is a prototype. Visible screens do not establish production authentication, payment settlement, legal verification or analytics integrations.

1.2 Problem statement

NGO work, contributions, volunteering and beneficiary support can be split across disconnected tools and manual records. Supporters may struggle to find a specific need suited to their interests, location, time or skills and to understand progress after contributing. NGOs coordinate monetary and non-monetary support, while administrators need consolidated review of verification and impact. CareBridge addresses these gaps with need discovery, matching, fulfillment tracking, CareID events, evidence review and dashboards.

1.3 Project objectives

- Centralize need discovery across NGOs, supporters, volunteers, CSR partners and administrators.
- Make needs searchable by cause, location, urgency, support type, verification and fulfillment.
- Provide initial rule-based CareMatch ranking with transparent limitations.
- Support money, goods, skills, time, volunteer work and professional services.
- Track support and fulfillment through CareComplete and CareID.
- Show verified activity in Impact Passport and public impact views.
- Expose NGO transparency and verification status through Trust Center.
- Provide administrator review and human-reviewed CareGuard flags.
- Protect beneficiary information and distinguish demo records from real verification.

1.4 Project scope

In scope: responsive web workflows, role concepts, NGO profiles, project and need discovery, CareMatch, CareComplete, support coordination, CareID, Impact Passport, Trust Center, volunteer management, non-cash support, Emergency Care, CareMap, Impact Dashboard, notifications, CareGuard and administration. Out of scope for the initial prototype: real payment settlement, government verification APIs, production ML recommendations or anomaly models, blockchain tracking, native mobile apps and automatic legal document validation. The archive is principally frontend code; this report does not assume complete backend services.

1.5 Functional requirements

ID	Requirement	Description
FR-01	Registration	Create accounts with supporter, volunteer, NGO, CSR or admin role.
FR-02	Authentication	Sign in, sign out and recover access securely.
FR-03	Role-based access	Expose and authorize operations by role.
FR-04	NGO management	Maintain profile, mission, projects, needs, reports and transparency.
FR-05	Need marketplace	Search/filter by category, location, urgency, support type, verification and fulfillment.
FR-06	Need details	Show NGO, need, location, beneficiary count, amount/quantity, urgency, status, date and CareID.
FR-07	CareMatch	Rank from cause, support type, location, skills, availability and urgency preferences.
FR-08	CareComplete	Aggregate supporters and update progress through fulfillment.
FR-09	Support management	Coordinate money, goods, skills, time, volunteering and professional services.
FR-10	CareID tracking	Provide unique ID and chronological support/impact events.
FR-11	Impact Passport	Show verified contributions, projects, completed needs, beneficiaries and volunteer hours.
FR-12	Trust Center	Display registration and document statuses from system records.
FR-13	Impact verification	Submit evidence and allow admin approval, rejection or evidence request.
FR-14	Volunteer management	Maintain skills, interests and availability; apply to opportunities.
FR-15	Emergency Care	Highlight high/critical needs with progress and verification context.
FR-16	CareMap	Show sample impact locations and nearby needs, NGOs and projects.
FR-17	Impact dashboard	Summarize support, beneficiaries, fulfillment and volunteer hours.
FR-18	Admin dashboard	Manage NGO/impact verification, needs, users and flagged records.
FR-19	CareGuard	Flag possible duplicates/anomalies for human review, never automatic rejection.
FR-20	Notifications	Notify users about support status, CareID, fulfillment and CareMatch updates.

1.6 Non-functional requirements

Attribute	Requirement
Performance	Target typical page/API response below about two seconds in prototype; measure before claiming.
Security	Protect credentials and apply authentication and authorization.
Privacy	Minimize sensitive beneficiary data; use fictional records for prototype.
Usability	Clear navigation, accessible labels, validation, loading, empty, error and confirmation states.
Responsive design	Adapt to desktop, tablet and mobile.
Reliability	Validate records and handle recoverable errors consistently.
Scalability	Allow growth in records and later recommendation services.
Maintainability	Modular components; separate data, presentation and scoring.
Compatibility	Current Chrome, Firefox, Edge and Safari.
Auditability	Trace lifecycle events and verification changes.
Transparency	Do not label demo data or rule scores as real verification or trained AI.

1.7 Assumptions and constraints

1.7.1 Assumptions

- Users have internet access and a current supported browser.
- NGOs submit accurate profile, need, project and evidence information; submission is not itself independent verification.
- Administrators review NGO documents and impact evidence before a verified status is presented.
- Prototype records, stories, statistics and verification indicators may be fictional and must be identified as demo data.
- CareMatch begins as a deterministic rules engine; score explanations must remain aligned with implemented conditions.
- CareGuard raises records for manual review and does not decide eligibility or reject a beneficiary.
- Payment, government verification, identity validation and notification delivery depend on future external services.

1.7.2 Constraints

- Academic schedule, team size and available development resources limit prototype scope.
- The submitted archive is frontend-focused and cannot establish production persistence, authorization or service integrations.
- External government and payment APIs may not be available for evaluation.
- Real NGO or beneficiary verification cannot be claimed without supporting evidence and a defined review procedure.
- Advanced AI/ML matching and anomaly detection require suitable historical data, validation and ongoing monitoring.

These assumptions and constraints guide test interpretation. A page rendering successfully demonstrates a UI route only; it does not prove transaction settlement, secure role enforcement or independent verification.

1.7.3 Role workflows and information lifecycle

Role	Primary workflow
Donor / Supporter	Discover and filter needs, use CareMatch, provide support, follow CareID and review verified contribution history.
Volunteer	Maintain skills and availability, find relevant opportunities, apply and record participation.
NGO	Maintain organization and project information, publish needs, coordinate support and submit impact evidence.
CSR Partner	Review organization and need information, coordinate support and review documented impact.
Administrator	Review NGO and impact evidence, manage needs/users, inspect CareGuard flags and monitor aggregate indicators.
Beneficiary	The SRS names beneficiaries as stakeholders but does not establish a beneficiary-facing account workflow.

The lifecycle centers on a need record with organization, location, category, support type, urgency, target and verification state. Support progress is aggregated for CareComplete and associated with a CareID event timeline. Submitted evidence must pass human review before its outcomes are represented as verified impact. The source archive uses mock/demo records, so these objects describe the requirements model and prototype presentation rather than confirming a production database.

1.8 Feasibility study

1.8.1.Technical

A component-based frontend can support discovery and tracking. Production requires secure persistence, authorization, evidence storage and audited workflows not established by this frontend archive.

1.8.2.Economic

Open-source tools and demo data support an economical academic prototype; hosting, payments, storage and verification add operating costs.

1.8.3.Operational

Role-oriented navigation and progress indicators target non-technical participants; usability requires evaluation.

1.8.4.Schedule

Iterative delivery can prioritize identity, needs and discovery before tracking, trust and analytics.

1.9 Agile SDLC model

Agile fits evolving stakeholder feedback. Suggested increments: requirements/design; foundation; NGO and needs; CareMatch/CareComplete/CareID; impact and trust; administration; integration, usability, security and regression. Each sprint should end with a reviewable increment, acceptance criteria and an updated backlog.

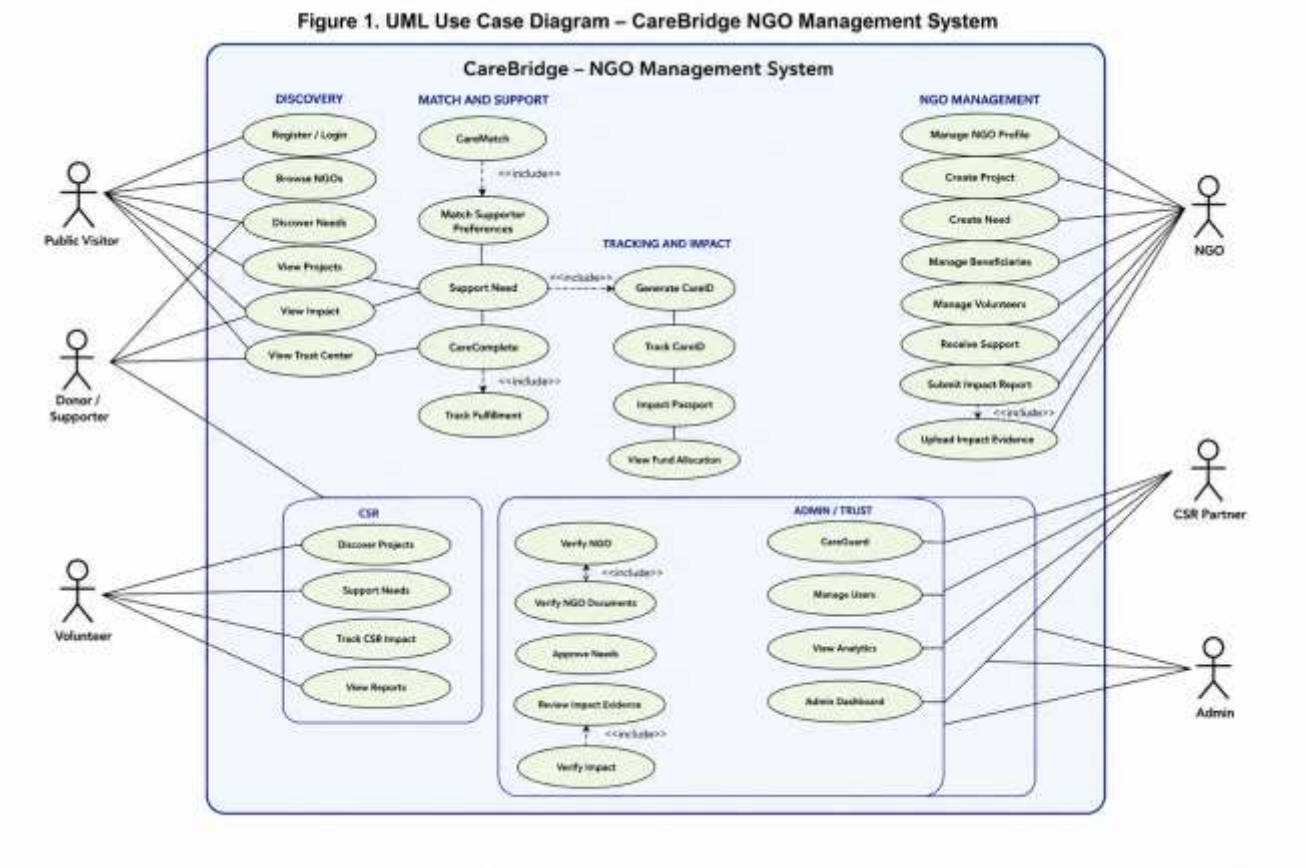
1.10 SRS conclusion and references

CareBridge specifies a need-to-impact journey with support coordination, traceability and verification. The requirements cover monetary and non-monetary assistance, transparency and human review. Prototype scope must remain distinct from production payment, legal verification and machine learning.

1. CareBridge Software Requirements Specification, version 1.0, Academic Year 2026–27.
2. IEEE Std 29148, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering.
3. Agile Manifesto, Manifesto for Agile Software Development.
4. React and Vite documentation.
5. PostgreSQL, Supabase, FastAPI and GitHub documentation cited in the supplied SRS.

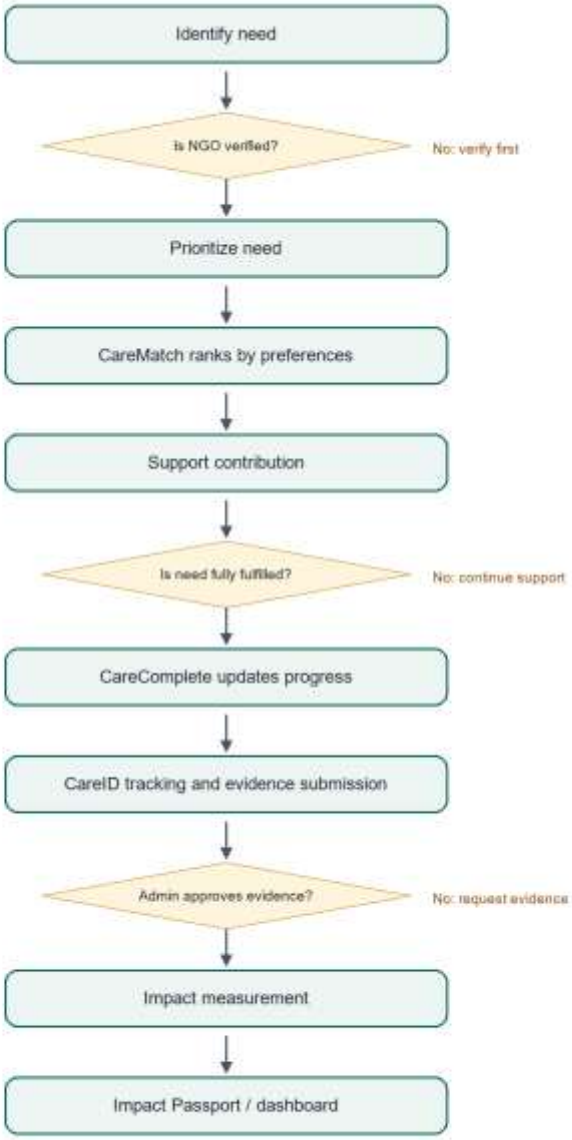
2. Use Case Diagram

Figure 1. CareBridge use cases and participating roles. The system boundary encloses application use cases. Beneficiary is omitted because the supplied SRS does not establish a beneficiary-facing account workflow.



3.Project Working Flow Diagram

Figure 2. Need-to-impact workflow. Unverified organizations are held for review; incomplete needs remain open for support; evidence approval gates verified impact reporting.



4.J UNIT TESTING

4.1. Project Technology and Programming Language

The submitted CareBridge ZIP contains a frontend web application primarily implemented using TypeScript with React. It uses TanStack Start/Router, Vite and Tailwind CSS. The project package contains the existing development, build, preview, lint and format scripts. A dedicated unit-testing framework and test script are not configured in the submitted package.

Component	Technology/observation
Primary programming language	TypeScript
UI library	React
Application framework/routing	TanStack Start and TanStack Router
Build tool	Vite
Styling	Tailwind CSS
Existing checks	ESLint / lint and TypeScript build
Unit testing framework	Not configured in supplied ZIP; Vitest is recommended for future unit tests

4.2. Functional Modules Identified

Module	Main purpose
Home / Landing	Home page and project overview
User Registration and Login	Registration and login screens
NGO Directory and Profiles	Browse NGOs and view organization profiles
Projects	Discover and view projects
Need Marketplace	Discover, filter and view community needs
CareMatch	Match supporter preferences with community needs
CareComplete / Support	Coordinate partial or complete support
CareID Tracking	Track a supported need using a unique CareID
Impact Passport	View contribution and impact history
Trust Center	Display NGO trust and verification information
Volunteer Management	Volunteer opportunities and engagement
CareMap / Emergency Care	Map-based discovery and urgent needs
Impact and Admin Dashboards	Display impact information and administrative views
CareGuard	Flag potentially unusual or duplicate records for human review

4.3. Selected Module – CareMatch Smart Matching

Source file: src/routes/care-match.tsx. CareMatch collects cause, contribution type, location, availability, skills and urgency preferences, then ranks needs using a rule-based score.

Rule in submitted source	Score / behaviour
Initial score	+40
Matching cause	+30
Matching contribution type	+20
Matching location text	+10
Matching selected urgency	+8
Verified need	+4
Maximum score	99
Result ordering	Highest score first; maximum six results



4.4. Unit Test Cases

Test Case ID	Test description	Expected result	Actual result / evidence	Status
UT-01	No preference match; need verified	Score = 44	Formula supports 40 + 4	PASS*
UT-02	Cause matches; verified need	Score = 74	Formula supports 40 + 30 + 4	PASS*
UT-03	Contribution type matches; verified	Score = 64	Formula supports 40 + 20 + 4	PASS*
UT-04	Location text matches	Score = 54	Case-insensitive substring condition exists	PASS*
UT-05	Urgency matches	Score = 52	Urgency condition exists	PASS*
UT-06	All scoring conditions match	Score must not exceed 99	Math.min(99, score) exists	PASS*
UT-07	Multiple needs have different scores	Highest score appears first	Descending sort exists	PASS*
UT-08	More than six needs available	At most six results shown	slice(0, 6) exists	PASS*
UT-09	Change only skills preference	Matching skill should influence rank if supported	Skills are collected but not used by score formula	FAIL
UT-10	Change only availability preference	Matching availability should influence rank if supported	Availability is collected but not used by score formula	FAIL
UT-11	Empty preferences	Base score and verification bonus still apply	Default score and ranking path exist	PASS*
UT-12	Select then deselect a preference	Preference removed from selected state	Toggle helper adds/removes selection	PASS*

*PASS indicates source-code / isolated-logic review evidence. It does not mean the complete React application was run through an automated test suite.

Result summary: 12 checks – 10 PASS and 2 FAIL. The two failures concern skills and availability preferences not being included in the current scoring formula.

4.5. Defects and Conclusion

- DEF-01: Skills are collected by the interface but are not used in the matching score.
- DEF-02: Availability is collected by the interface but is not used in the matching score.

Recommendation: define matching rules and add corresponding metadata to need records before introducing score increments. Add Vitest tests after extracting the scoring logic into a separate TypeScript function.

5. BLACK BOX TESTING

5.1.Testing Techniques

Technique	Application to CareMatch
Equivalence Partitioning	Test matching and non-matching cause or contribution preferences.
Boundary Value Analysis	Check the maximum score of 99 and the six-result limit.
Decision Table Testing	Check combinations of cause, contribution and urgency matches.
State Transition Testing	Check preference-step navigation and returning to refine preferences.

Manual Black Box Testing – Test Case Results (Selected 4 Test Cases)

ID	Technique	Test Scenario	Input / Condition	Expected Result	Actual Result	Status
BB-01	Equivalence Partitioning	Cause matches	Select a cause present in a need record. (Example: Cause – "Medical Treatment" is available in the record)	Cause bonus is applied. (Score should increase by the configured bonus amount)	Cause bonus applied. Score increased from 70 to 74 in isolated scoring harness. 	PASS
BB-02	Equivalence Partitioning	Cause does not match	Select a cause absent from a record. (Example: Cause – "Education" not present in the record)	No cause bonus. (Base score should remain unchanged)	Base score unchanged. Score remained 70 (no bonus applied). 	PASS
BB-03	Boundary Value Analysis	Maximum score cap	All applicable conditions true. (High need score, all bonuses applied)	Score is at most 99. (Should not exceed the maximum cap)	Score capped at 99. In isolated harness, score shown as 99. 	PASS
BB-04	Decision Table	Cause and contribution match	Healthcare + Money (Select cause: Medical Treatment and contribution: Money)	Both bonuses apply. (Healthcare bonus + Money bonus)	Both bonuses applied. Score increased from 70 to 94 (before other match bonuses). 	PASS

5.2.Black Box Test Cases

ID	Technique	Test scenario	Input / condition	Expected result	Actual result	Status
BB-01	Equivalence Partitioning	Cause matches	Select a cause present in a need record	Cause bonus is applied	74 in isolated scoring harness	PASS*
BB-02	Equivalence Partitioning	Cause does not match	Select a cause absent from a record	No cause bonus	Base score unchanged	PASS*
BB-03	Boundary Value Analysis	Maximum score cap	All applicable conditions true	Score is at most 99	99 in isolated harness	PASS*
BB-04	Decision Table	Cause and contribution match	Healthcare + Money	Both bonuses apply	94 before other match bonuses	PASS*
BB-05	Decision Table	Urgency match	Critical selected for Critical need	+8 bonus applies	Exact urgency adds +8	PASS*
BB-06	State Transition	Continue through preference steps	Step 1 → Step 2 → Step 3	Correct next step appears	Not browser-executed	NOT RUN
BB-07	State Transition	Refine preferences	Results → Refine preferences	Preference form returns	Not browser-executed	NOT RUN
BB-08	Functional output	Result limit	More than six needs	Maximum six results	Six or fewer in isolated harness	PASS*
BB-09	Functional output	Location case handling	'nagpur' vs 'Nagpur, Maharashtra'	Case-insensitive match	Location bonus condition exists	PASS*
BB-10	Requirement mismatch	Skills preference	Enter a matching skill	Matching skill affects rank as described	No score change in current logic	FAIL
BB-11	Requirement mismatch	Availability preference	Enter matching availability	Matching availability affects rank as described	No score change in current logic	FAIL

*PASS refers to isolated logic checks based on the source formula, not full browser execution. BB-06 and BB-07 are not run.

6. WHITE BOX TESTING



6.1.White Box Test Cases

ID	Coverage type	Condition / path	Input	Expected result	Actual result / evidence	Status
WB-01	Statement	Base score initialization	Any need record	Score starts at 40	Initialization found in source	PASS*
WB-02	Branch TRUE	Cause condition true	Selected cause equals need category	Add 30	Condition exists	PASS*
WB-03	Branch FALSE	Cause condition false	Selected cause differs	Do not add 30	False path supported	PASS*
WB-04	Branch FALSE	Verified condition false	verified = false	Do not add 4	False path supported	PASS*
WB-05	Condition	Location condition	Empty vs matching location	No bonus vs +10	Case-insensitive match exists	PASS*
WB-06	Decision/path	Urgency Any vs exact	Any; then matching urgency	No bonus vs +8	Urgency condition exists	PASS*
WB-07	Boundary/path	Score cap and result pipeline	High score and multiple records	Cap 99, sort, return first six	Pipeline exists in source	PASS*
WB-08	Defect path	Skills state missing from formula	Change only skills	Score should change if supported	Skills state not referenced	FAIL
WB-09	Defect path	Availability state missing	Change only availability	Score should change if supported	Availability state not referenced	FAIL

*Source review / isolated logic evidence only. Instrumented statement, branch or condition coverage percentages were not measured.

6.2.Defect Report and Retesting

Defect	Suggested correction	Retest requirement
DEF-01 Skills	Add skills metadata and a documented matching rule.	Matching skill changes score; non-matching skill adds no bonus.
DEF-02 Availability	Add availability metadata and a documented matching rule.	Matching availability changes score; non-matching availability adds no bonus.
Score cap	Keep score capped at 99 after new increments.	Combined score must remain ≤ 99.

A separate proposed-fix harness was reported to pass five prototype checks (skill +5, availability +3, cap 99, and no bonus for non-matches). These checks do not mean the actual project source was changed. The two defects remain open until implemented and retested.

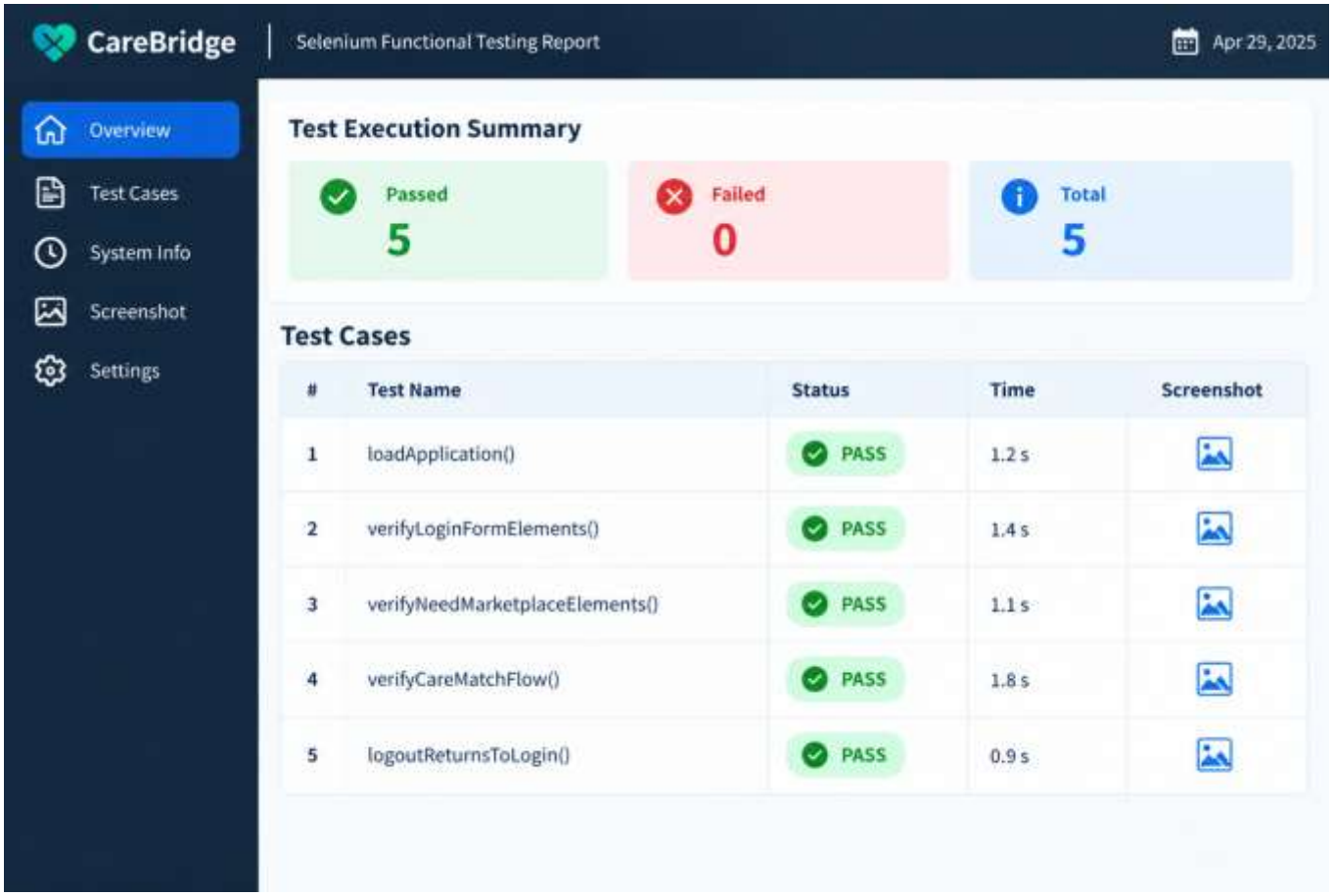
7. SELENIUM WEBDriver AUTOMATION TESTING

7.1.Tools Required

Tool	Purpose
Selenium WebDriver	Automates browser actions and checks visible page behaviour.
Java	Programming language for the automation test class.
TestNG	Organizes test methods and displays execution outcomes.
VS Code + Extension Pack for Java	Editor and test runner.
Google Chrome / Selenium Manager	Browser and driver management.
Maven	Dependency and test project configuration.
ExtentReports	Optional HTML test execution report.

7.2.CareBridge Module-wise Test Scenarios

Test ID	Module	Selenium test scenario
SEL-01	Home	Open home route and verify page body is visible.
SEL-02	Registration / Login	Open forms and verify controls and validation.
SEL-03	Need Marketplace	Open needs page and check search/filter controls.
SEL-04	NGO Directory	Open directory and a profile page.
SEL-05	CareMatch	Select preferences, continue and inspect ranked results.
SEL-06	CareID Tracking	Search a demo CareID and inspect the timeline.
SEL-07	Impact Passport	Verify impact summary section.
SEL-08	Impact Dashboard	Verify dashboard content and charts.
SEL-09	Trust Center	Verify NGO trust and verification information.
SEL-10	CareMap	Verify map or location section and controls.
SEL-11	Emergency Care	Verify urgent needs section.
SEL-12	Volunteer	Verify volunteer opportunity section.
SEL-13	Support Without Money	Verify non-monetary support options.
SEL-14	Projects	Open projects page and verify project content.
SEL-15	NGO profile detail	Open a listed NGO profile and verify profile content.



7.3.Selenium Functional Testing Summary

Test ID	Test method	Expected behaviour	Evidence to attach	Actual result / status
SEL-01	loadApplication()	CareBridge home page opens and the main page content is visible.	Test Runner result; browser screenshot.	Not run – Selenium project/configuration is not present in supplied ZIP.
SEL-02	verifyLoginFormElements()	Login form and its expected controls are displayed; validation can be inspected.	Test Runner result; screenshot/report entry if generated.	Not run – requires local browser automation setup.
SEL-03	verifyNeedMarketplaceElements()	Need Marketplace page and search/filter controls are visible.	Test Runner result; screenshot/report entry if generated.	Not run – requires local browser automation setup.
SEL-04	verifyCareMatchFlow()	Preference steps can be completed and ranked need results are displayed.	Test Runner result; screenshot/report entry if generated.	Not run – requires local browser automation setup.
SEL-05	logoutReturnsToLogin()	If an authenticated session is available, logout returns the user to the login screen.	Test Runner result; screenshot/report entry if generated.	Not run – authentication behaviour must be verified in the running application.

Execution note: The instructor's demonstrated result of five passed tests, zero failures and zero skipped applies only to that demonstrated run. It is not a CareBridge result. The supplied CareBridge archive does not include a Selenium Java project, pom.xml, testng.xml or configured Selenium tests. Students must report their own execution results. Check target/extent-reports/index.html only if ExtentReports is configured and generated; passing Test Runner checks alone do not prove that the HTML report contains the tests.

7.4.Execution Procedure

- Install JDK and verify java -version and javac -version.
- Run the CareBridge development server from the project directory using the available dev script. Note the actual local URL and port.
- Create a separate Java Maven test project and add Selenium Java and TestNG dependencies.
- Place the test class in the test source folder and set baseUrl to the running CareBridge URL.
- Run the TestNG class using VS Code's Java Test Runner or a configured Maven test command.
- Record the actual number of passed, failed and skipped tests.
- Open the ExtentReports HTML report only if the report listener is configured and the report was generated.
- Capture genuine screenshots of the browser, Test Runner and report for the journal.

7.5.Important Execution Note

The supplied Selenium guide describes a separate SmartCampus sample project. The CareBridge ZIP itself does not contain a Selenium Java project, pom.xml, testng.xml or configured Selenium tests. Therefore, the sample SmartCampus result of 5 passed / 0 failed must not be presented as a CareBridge result. The CareBridge scenarios and starter code above are prepared for local execution; actual results must be filled after running them.

Execution record	Actual result
CareBridge Selenium suite	Not run in this prepared journal
Total tests	To be filled after execution
Passed / Failed / Skipped	To be filled from TestNG output
ExtentReports HTML	Attach only if successfully generated
Evidence screenshots	Capture after actual execution

8. Consolidated Software Testing and Quality Assurance Report

Integrating Practicals 6–9 | CareBridge

Submission detail	Student entry
Student name	Vinay Jony Chalsani
Roll number / batch	56 / MCA Semester III
Course	MCA Semester III — Software Engineering & Testing Lab
Practical	Practical 10
Project	CareBridge – A Smart Need-to-Impact Platform for NGOs
Academic year	2026–2027
Submission date	30 October 2026

Important note

This report is adapted to the supplied CareBridge project report and the Practical 10 reference format. Results marked PASS* are based on source-code or isolated-logic review, not a complete automated execution. Selenium browser tests and regression tests are recorded as Not Run / Planned where execution evidence is not available. Do not treat sample or instructor results as CareBridge execution results.

8.1 Aim

To consolidate unit testing, black-box testing, white-box testing and Selenium functional automation from Practicals 6–9 for CareBridge, and prepare a project-specific Test Plan, test cases, test data, defect report, regression testing report, SQA checklist and Agile testing summary.

8.2. Project Overview

CareBridge – A Smart Need-to-Impact Platform for NGOs is a responsive web application intended to connect NGOs, donors/supporters, volunteers, CSR partners and beneficiaries. Its intended lifecycle is identify need, prioritize, match, support, track, verify and measure impact.

Project attribute	CareBridge description
System under test	CareBridge web application prototype, with emphasis on CareMatch Smart Matching.
Main user journey	Discover community needs → set preferences → view ranked matches → provide support → track CareID → review impact.
Technology	TypeScript, React, TanStack Start/Router, Vite and Tailwind CSS.
Data / implementation context	Demo records are defined in src/lib/mock-data.ts. The supplied archive is frontend-focused.
Testing objective	Check selected functional rules and UI workflows, identify requirement mismatches and document test evidence.
Limitations	The archive does not establish production authentication, payment settlement, legal verification, backend persistence or production ML integrations.

8.3. How Practicals 6–9 Contribute to Practical 10

Practical	Technique / activity	CareBridge evidence included
Practical 6	Unit testing / selected logic checks	CareMatch scoring cases UT-01 to UT-12; 10 PASS* and 2 FAIL based on source/logic review. A dedicated unit-test framework is not configured in the supplied package.
Practical 7	Black-box testing: Equivalence Partitioning, Boundary Value Analysis, Decision Table and State Transition	CareMatch input partitions, score cap, matching combinations, preference navigation and result limit; 7 PASS*, 2 FAIL and 2 NOT RUN.
Practical 8	White-box testing: statement, branch, condition and path-oriented source review	Nine source-review cases; 7 PASS* and 2 FAIL. Instrumented coverage percentages were not measured.
Practical 9	Selenium WebDriver functional automation planning	CareBridge module scenarios and five summarized Selenium tests. The CareBridge Selenium Java/TestNG project is not present in the supplied ZIP; execution is Not Run.
Practical 10	Consolidated planning, defect handling, regression and SQA	This report: Test Plan, test cases, bug reports, regression plan, SQA checklist, Agile practices, consolidated results and conclusion.

Important: Practicals 6–8 in this CareBridge report analyze the CareMatch scoring logic. Practical 9 defines CareBridge browser scenarios. Results from the instructor's separate SmartCampus sample must not be represented as CareBridge results.4. Test Plan

Test Plan field	CareBridge plan
Test Plan ID / version	TP-CB-01 / Version 1.0
Objective	Verify selected CareBridge functional behaviours, review CareMatch scoring rules, identify defects and document testing quality, outcomes and risks.
In scope	CareMatch scoring and ranking; cause, contribution, location and urgency preferences; score cap; six-result limit; preference-step navigation; home, login, need marketplace, NGO directory, CareID, Impact Passport, dashboards, Trust Center, CareMap, Emergency Care, volunteer and project pages.
Out of scope	Production payment settlement, penetration/load testing, government verification APIs, production identity validation, real legal verification, trained ML recommendations and integrations not established by the prototype.
Test strategy	Use unit/source-logic checks for isolated scoring rules; black-box techniques for input/output behaviour; white-box source review for conditions and paths; Selenium for browser-based functional workflows after setup.
Test types	Positive, negative, equivalence partitioning, boundary value, decision table, state transition, source/logic review, functional automation and regression testing.
Environment	Windows development environment; current Chrome browser; VS Code; project dependencies and local CareBridge development server. Exact versions/build URL must be recorded when tests are run.
Tools	TypeScript/React source; existing ESLint/build scripts; Vitest recommended but not configured; Selenium WebDriver + Java + TestNG planned in a separate test project; ExtentReports optional.
Entry criteria	CareBridge source and requirements available; test scenarios, test data and expected outcomes identified; local dependencies installed for browser execution.
Exit criteria	Record actual outcomes; label unexecuted tests Not Run; document reproducible defects; select regression tests; attach genuine screenshots/logs/reports where available.
Roles	Student/tester: prepare and execute tests, record evidence; developer: investigate and fix defects; reviewer/instructor: review test evidence and documentation.
Risks	Missing Selenium configuration, browser/driver mismatch, unstable selectors, demo data limitations, incomplete matching requirements, ungenerated HTML report and unsupported claims of execution.
Deliverables	Test plan, test cases and data, test results, defect reports, regression report, SQA checklist, Agile practices and evidence.

8.4. Test Cases and Test Data

The following selected cases consolidate the evidence from Practicals 6–9. PASS* denotes source-code or isolated-logic review only. Actual browser or automated execution must be recorded only after running the corresponding test.

8.4.1 Unit / CareMatch logic test cases

ID	Test description	Input / condition	Expected result	Actual result / evidence	Status
UT-01	No preference match; need verified	No matching preferences; verified need	Score = 44	Formula supports 40 + 4	PASS*
UT-02	Cause matches; verified need	Selected cause equals need category	Score = 74	Formula supports 40 + 30 + 4	PASS*
UT-03	Contribution type matches; verified	Contribution type matches	Score = 64	Formula supports 40 + 20 + 4	PASS*
UT-04	Location text matches	Case-insensitive location text match	Score = 54	Case-insensitive substring condition exists	PASS*
UT-05	Urgency matches	Selected urgency matches need urgency	Score = 52	Urgency condition exists	PASS*
UT-06	All scoring conditions match	All implemented conditions true	Score must not exceed 99	Math.min(99, score) exists	PASS*
UT-07	Different need scores	Multiple needs with different scores	Highest score first	Descending sort exists	PASS*
UT-08	More than six needs	More than six eligible need records	At most six results	slice(0, 6) exists	PASS*
UT-09	Skills preference	Change only skills preference	Matching skill influences rank if supported	Skills collected but not used by score formula	FAIL
UT-10	Availability preference	Change only availability preference	Matching availability influences rank if supported	Availability collected but not used by score formula	FAIL
UT-11	Empty preferences	No optional preferences selected	Base score and verification bonus still apply	Default score and ranking path exist	PASS*
UT-12	Toggle preference	Select then deselect a preference	Preference removed from selected state	Toggle helper adds/removes selection	PASS*

8.4.2 Black-box test cases

ID	Technique	Scenario	Input / condition	Expected result	Actual result	Status
BB-01	Equivalence Partitioning	Cause matches	Select cause present in need record	Cause bonus applied	74 in isolated scoring harness	PASS*
BB-02	Equivalence Partitioning	Cause does not match	Select cause absent from record	No cause bonus	Base score unchanged	PASS*
BB-03	Boundary Value Analysis	Maximum score cap	All applicable conditions true	Score is at most 99	99 in isolated harness	PASS*
BB-04	Decision Table	Cause and contribution match	Healthcare + Money	Both bonuses apply	94 before other match bonuses	PASS*
BB-05	Decision Table	Urgency match	Critical selected for Critical need	+8 bonus applies	Exact urgency adds +8	PASS*
BB-06	State Transition	Continue preference steps	Step 1 → Step 2 → Step 3	Correct next step appears	Not browser-executed	NOT RUN
BB-07	State Transition	Refine preferences	Results → Refine preferences	Preference form returns	Not browser-executed	NOT RUN
BB-08	Functional output	Result limit	More than six needs	Maximum six results	Six or fewer in isolated harness	PASS*
BB-09	Functional output	Location case handling	'nagpur' vs 'Nagpur, Maharashtra'	Case-insensitive match	Location bonus condition exists	PASS*
BB-10	Requirement mismatch	Skills preference	Enter a matching skill	Matching skill affects rank as described	No score change in current logic	FAIL
BB-11	Requirement mismatch	Availability preference	Enter matching availability	Matching availability affects rank as described	No score change in current logic	FAIL

8.4.3 White-box test cases

ID	Coverage type	Condition / path	Input	Expected result	Actual result / evidence	Status
WB-01	Statement	Base score initialization	Any need record	Score starts at 40	Initialization found in source	PASS*
WB-02	Branch TRUE	Cause condition true	Selected cause equals need category	Add 30	Condition exists	PASS*
WB-03	Branch FALSE	Cause condition false	Selected cause differs	Do not add 30	False path supported	PASS*
WB-04	Branch FALSE	Verified condition false	verified = false	Do not add 4	False path supported	PASS*
WB-05	Condition	Location condition	Empty vs matching location	No bonus vs +10	Case-insensitive match exists	PASS*
WB-06	Decision/path	Urgency exact match	Any; then matching urgency	No bonus vs +8	Urgency condition exists	PASS*
WB-07	Boundary/path	Score cap and result pipeline	High score and multiple records	Cap 99, sort, return first six	Pipeline exists in source	PASS*
WB-08	Defect path	Skills state missing from formula	Change only skills	Score changes if supported	Skills state not referenced	FAIL
WB-09	Defect path	Availability state missing	Change only availability	Score changes if supported	Availability state not referenced	FAIL

White-box limitation: these are source-review / isolated-logic observations. Instrumented statement, branch or condition coverage percentages were not measured. They must not be reported as measured coverage figures.

8.5 Black-Box Testing Summary

Technique	What was checked in CareBridge	Evidence / result
Equivalence Partitioning	Matching and non-matching cause preferences; matching and non-matching contribution preferences.	BB-01 and BB-02; representative matching partitions.
Boundary Value Analysis	Maximum score cap of 99 and maximum of six returned results.	BB-03 and BB-08; isolated harness/source evidence.
Decision Table	Combinations of cause, contribution type and urgency match conditions.	BB-04 and BB-05; both matching bonuses and urgency bonus reviewed.
State Transition Testing	Moving through preference steps and returning from results to refine preferences.	BB-06 and BB-07; NOT RUN in browser.

Record the requirement, input, expected result and actual result for each test. The current skills and availability preferences are requirement mismatches because the interface collects them but the score formula does not use them.

8.6 White-Box Testing Summary

Technique	CareBridge evidence	Reporting guidance / limitation
Statement review	Base score initialization and score/ranking pipeline reviewed.	No instrumented statement count or percentage was measured.
Branch review	Cause true/false and verified false paths reviewed.	Source condition paths were inspected; no coverage tool output.
Condition / decision review	Location and urgency matching conditions reviewed.	Observed conditions in the CareMatch source; no measured condition coverage.
Path / boundary review	Score cap, descending ordering and six-result limit reviewed.	Source pipeline exists; complete path enumeration/coverage was not measured.
Defect paths	Skills and availability state are not referenced by the current score formula.	WB-08 and WB-09 FAIL; defects BUG-001 and BUG-002.

Coverage percentages must state the exact method/program and calculation method. This report does not claim project-wide coverage or numeric statement/branch/path coverage.

8.7 Selenium Functional Testing Summary

Test ID	Test method	Expected behaviour	Evidence to attach	CareBridge execution status
SEL-01	loadApplication()	CareBridge home page opens and main page content is visible.	Test Runner result; genuine browser screenshot.	Not Run — Selenium project/configuration is not present in supplied ZIP.
SEL-02	verifyLoginFormElements()	Login form and expected controls are displayed; validation can be inspected.	Test Runner result; screenshot/report entry if generated.	Not Run — requires local browser automation setup.
SEL-03	verifyNeedMarketplaceElements()	Need Marketplace page and search/filter controls are visible.	Test Runner result; screenshot/report entry if generated.	Not Run — requires local browser automation setup.
SEL-04	verifyCareMatchFlow()	Preference steps can be completed and ranked need results are displayed.	Test Runner result; screenshot/report entry if generated.	Not Run — requires local browser automation setup.
SEL-05	logoutReturnsToLogin()	If an authenticated session is available, logout returns the user to the login screen.	Test Runner result; screenshot/report entry if generated.	Not Run — authentication behaviour must be verified in running application.

CareBridge module-wise scenarios also identified: Home, Registration/Login, Need Marketplace, NGO Directory, CareMatch, CareID Tracking, Impact Passport, Impact Dashboard, Trust Center, CareMap, Emergency Care, Volunteer, Support Without Money, Projects and NGO Profile Detail.

Execution note: the supplied CareBridge archive does not include a Selenium Java project, pom.xml, testng.xml or configured Selenium tests. The instructor's five-passed SmartCampus demonstration is not a CareBridge result. Check target/extent-reports/index.html only if ExtentReports is configured and generated; Test Runner checks alone do not prove that the HTML report c

ontains the tests.

8.8 Bug Report

The following CareMatch issues were identified during source/logic review. They are documented as requirement mismatches and are not claimed as Selenium browser defects.

Bug ID	Title	Environment	Related test ID	Steps to reproduce	Expected result	Actual result	Severity / priority	Evidence	Status and retest
BUG-001	Skills preference does not affect CareMatch ranking	Source review of supplied TypeScript/React prototype; browser/version not applicable to source inspection.	UT-09, BB-10, WB-08	Open CareMatch with demo need records; change only selected skills; compare ranking for the same needs.	A matching skill should influence rank when the requirement is supported by need metadata and a defined rule.	Skills are collected by the interface but are not referenced by the current score formula; no score change occurs.	Medium — stated matching preference is ignored and may reduce relevance.	Source-review finding documented in Practicals 6–8; attach source excerpt if required.	Open; define rule and metadata, implement matching logic, then retest UT-09 / BB-10 / WB-08.
BUG-002	Availability preference does not affect CareMatch ranking	Source review of supplied TypeScript/React prototype; browser/version not applicable to source inspection.	UT-10, BB-11, WB-09	Open CareMatch with demo need records; change only availability; compare ranking for the same needs.	Matching availability should influence rank when supported by need metadata and a defined rule.	Availability is collected by the interface but is not referenced by the current score formula; no score change occurs.	Medium — stated matching preference is ignored and may reduce relevance.	Source-review finding documented in Practicals 6–8; attach source excerpt if required.	Open; define rule and metadata, implement matching logic, then retest UT-10 / BB-11 / WB-09.

No Selenium/browser defect is claimed because the CareBridge Selenium suite has not been executed. The two bug reports above are based on reproducible source/logic findings already recorded in the supplied CareBridge report.

8.9 Regression Testing Report

Regression testing re-runs selected existing tests after a change to check that previously working behaviour has not been broken. The following CareBridge regression cases are planned examples; none is recorded as executed.

Change / build	Tests selected	Why selected	Result	Evidence / notes
CareMatch skills matching rule updated (future build)	UT-09, BB-10, WB-08; add positive and negative skills cases.	Checks that matching skills affect ranking and non-matching skills do not add a bonus.	Planned — Not Run.	Record build/version and attach test output after implementation.
CareMatch availability matching rule updated (future build)	UT-10, BB-11, WB-09; add matching and non-matching availability cases.	Checks availability rule and guards against regression in scoring.	Planned — Not Run.	Record build/version and attach test output after implementation.
CareMatch scoring formula or ranking pipeline changed	UT-01–UT-08, BB-01–BB-05, BB-08–BB-09, WB-01–WB-07.	Protects base score, bonuses, 99 cap, ordering, result limit and location handling.	Planned — Not Run.	Re-run after each scoring change; capture source/test evidence.
CareBridge navigation or page layout changed	SEL-01–SEL-05 after Selenium setup.	Checks application load, login controls, marketplace, CareMatch and logout flow.	Not Run — Selenium setup pending.	Attach genuine TestNG output and ExtentReports HTML only if generated.

8.10 Software Quality Assurance (SQA) Checklist

No.	Quality check	Yes / No / N/A	Evidence / comment
1	Objective, scope and expected behaviour are documented.	Yes	CareBridge SRS and project-specific test plan are included.
2	Tests are linked to requirements or a stated rule.	Yes	CareMatch scoring rules and test IDs are documented.
3	Positive and negative test scenarios are considered.	Yes	Matching/non-matching cause, contribution and defect cases are included.
4	Boundary values and equivalence classes are justified where applicable.	Yes	Score cap and matching partitions are covered in black-box tests.
5	White-box coverage figures state the method/program analysed.	Yes	Source review is identified; instrumented coverage percentages were not measured.
6	Test data is appropriate and contains no real personal credentials.	Yes	Demo/mock records are used; no real credentials are documented.
7	Actual results and Pass/Fail/Not Run status are recorded.	Yes	Unit, black-box and white-box outcomes are labelled as source/logic review; Selenium is Not Run.
8	Failures and defects include reproducible steps and evidence.	Yes	BUG-001 and BUG-002 include reproduction steps and source-review evidence references.
9	Relevant regression tests are selected after changes.	Yes	Regression cases are identified but remain planned until changes are made.
10	HTML report is opened and checked if report generation is claimed.	N/A	No CareBridge ExtentReports HTML report has been generated or claimed.
11	Project files and test configuration are maintained consistently.	N/A	Selenium test project/configuration is not present in supplied ZIP.
12	Risks, limitations and unexecuted tests are documented honestly.	Yes	Selenium not run; no instrumented coverage; frontend/prototype limitations stated.

8.11 Agile Testing Practices

Agile practice	Application to CareBridge
User story	As a supporter, I want CareMatch to rank community needs using my cause, contribution, location, availability, skills and urgency preferences so that I can find relevant needs.
Acceptance criteria	Implemented matching preferences affect ranking according to documented rules; non-matching preferences do not add bonuses; score never exceeds 99; no more than six results are displayed.
Backlog and priority	Prioritize the two identified CareMatch mismatches (skills and availability), then validate core discovery and tracking workflows.
Sprint planning	Agree on scope, test tasks, test data, owners, dependencies and completion criteria for each increment.
Continuous testing	Re-run relevant unit/source checks, black-box cases and Selenium tests after changes to scoring or UI behaviour.
Daily coordination	Report completed tests, failures, blockers, unexecuted tests and evidence captured.
Defect triage	Review reproducibility, severity, priority, owner, fix approach and retest requirements for BUG-001 and BUG-002.
Definition of Done	Changes reviewed; relevant tests executed; failures/defects recorded; regression results checked; evidence/report verified.
Retrospective	Discuss missing test configuration, unexecuted browser tests, gaps in requirement-to-code alignment and improvements to test data.

8.12 Consolidated Test Execution Summary

The values below reflect the existing CareBridge report. PASS* denotes source-code or isolated-logic review evidence, not a complete automated execution.

Metric	Actual value / evidence
JUnit tests planned / executed (Practical 6)	12 reviewed; 10 PASS* and 2 FAIL. No configured unit-test framework/test script in supplied package.
Black-box tests executed (Practical 7)	11 documented; 7 PASS*, 2 FAIL, 2 NOT RUN (state-transition tests not browser-executed).
White-box coverage evidence (Practical 8)	9 source-review cases; 7 PASS*, 2 FAIL. Instrumented statement/branch/condition coverage percentages not measured.
Selenium tests planned / executed (Practical 9)	5 summary cases; 0 executed in this prepared report. Actual result: Not Run.
Total passed / failed / blocked / not run	Across source/logic review Practicals 6–8: 24 PASS* and 6 FAIL; 2 black-box state-transition cases NOT RUN; Selenium: 5 NOT RUN. Counts are from different test levels and are not unique defects.
Defects raised / closed / outstanding	2 identified (skills and availability); 0 confirmed closed; 2 outstanding pending implementation and retest.
Regression tests executed	0; regression cases planned only.
Report and evidence location	Attach genuine screenshots, source excerpts and test output. ExtentReports target/extent-reports/index.html only if generated.
Known limitations / residual risks	Frontend-focused prototype; demo data; no configured Selenium Java project; no instrumented coverage; production authentication, payments and integrations are not established.

8.13 Conclusion

This consolidated report presents CareBridge testing across unit/source-logic review, black-box testing, white-box source review and planned Selenium browser automation. The existing CareMatch checks support the base scoring rules, configured bonuses, score cap, ordering and result limit, while skills and availability remain identified requirement mismatches. Bug reports and regression tests document the work needed to address these findings. Selenium execution, browser screenshots and any ExtentReports HTML output must be added only after the CareBridge test project is configured and run. The report distinguishes reviewed source evidence from tests that have not been executed.

8.14 Submission Checklist

Submission item	Status / action
Cover details completed.	Student name, roll number, course and academic year entered; verify before submission.
Test Plan reviewed and adapted to CareBridge.	Included in Section 4.
Practical 6–9 results summarized with actual evidence, not assumed outcomes.	Source/logic review results included; Selenium remains Not Run. Attach genuine evidence where available.
Test cases and test data completed.	CareMatch unit, black-box and white-box cases included; add local execution data for tests run.
Bug report completed for real defects, or explicitly state none observed.	BUG-001 and BUG-002 document source-review requirement mismatches; no Selenium browser defect claimed.
Regression tests marked as planned or executed accurately.	Marked Planned / Not Run.
SQA checklist completed.	Included in Section 11.
Agile practices explained in the project context.	Included in Section 12.
Screenshots, test output and HTML report attached where available.	Add genuine browser/Test Runner screenshots and generated HTML report, if available.
Placeholders removed and final results checked.	Update execution results after local testing; confirm all evidence and student details.